```
      post :create,
        params: {project: {name: "Runway", tasks: "start something:2"}}
      expect(CreatesProject).to have_received(:new)
        .with(name: "Runway", task_string: "start something:2")
    end

  end
end
```

On the first two lines the doubles are created. You need to do this in two steps because the controller both instantiates a CreatesProject object and calls create on it. The power of this test is in the mock expectations. You're validating all of the following:

- That the controller calls CreatesProject.new
- That the call passes a name and a task_string as key/value pairs based on the incoming parameters
- That the controller calls create on the return value of CreatesProject.new

That's a fairly detailed description of the controller's responsibilities.

This test makes only a minimal attempt to validate the objects outside of the controller. The use of instance_double to create the test double guarantees the test will fail if the controller calls a method on the action item that doesn't exist, but makes no other claim about what the action item does. That's great in that a bug in CreatesProject will trigger only one test failure, making it easier to track down. But it's terrifying in that it's possible for this test to pass without the underlying code working.

Because it is possible to have tests pass due only to mismatches between the API and the test double, generally I do this kind of testing based on setting mock expectations only if there is a separate integration test tying together all the small, focused unit tests.

A plus for this test, however, is speed. Since it doesn't contact the model layer, it's probably going to be fast.

A downside is readability. It can be hard to look at a mocked test and determine exactly what is being validated. Using spies can help here because spies force you to be explicit about the expectations you're claiming in the test at the end of the test, where you normally see test expectations.

Finally, an elaborate edifice of mocked methods runs the risk of causing the test to depend on very specific details of the method structure of the object being mocked. This can make the test brittle in the face of refactorings that